

```
alex.omegapy@MacStudio CSC510-CTAs % .venv/bin/python CTA_Module
3/handmade_shallow_ann.py
```

HAND-MADE SHALLOW ARTIFICIAL NEURAL NETWORK

This classroom program trains a small NumPy artificial neural network to estimate a fictional delivery duration. The point is not to model real traffic. The point is to make the learning process visible, one phase at a time.

The walkthrough is divided into readable sections. During an interactive run, the program pauses between sections so the equations and results do not arrive as one large wall of text.

Press Enter to prepare the training data...

1. PREPARE THE TRAINING MATRICES X AND Y

Where the examples come from

Before the network can learn, it needs paired examples. Each row of X describes one fictional route, and the matching row of Y contains the classroom target duration for that route.

```
X[i] = [x1_highway_miles, x2_local_road_miles, x3_congestion_flag]
Y[i] = 10 + (x1 / highway_speed) * 60 + (x2 / 20) * 60
highway_speed = 50 mph when x3 = 0; 25 mph when x3 = 1
local_road_speed = 20 mph when x3 = 0; 10 mph when x3 = 1
```

What the matrix dimensions mean

```
X.shape = (352, 3)
352 route examples, with 3 input features in each row
Y.shape = (352, 1)
352 matching targets, with 1 delivery duration in each row
```

Why the values are standardized

The raw distances and target minutes use different numerical scales. Standardization puts them on comparable scales before training. The same training-set means and standard deviations are later reused for user input.

```
X_standardized = (X - mu_X) / sigma_X
Y_standardized = (Y - mu_Y) / sigma_Y
```

Press Enter to inspect the network and its learning cycle...

2. SET UP THE NETWORK AND ITS FOUR LEARNING PHASES

Network architecture

```
X (m, 3) -> A1 (m, 6) -> Y_hat (m, 1)
```

```

Layer layout:                3 inputs -> 6 hidden neurons -> 1
output
W1 and b1 shapes:            (3, 6) and (1, 6)
    W1 connects the 3 inputs to all 6 hidden neurons
W2 and b2 shapes:            (6, 1) and (1, 1)
W2 and b2 shapes:            (6, 1) and (1, 1)
    W2 connects the 6 hidden activations to the single output
Hidden activation:            ReLU
Output activation:            linear identity
Loss function:                mean squared error J
Learning rate eta:            0.05
Training rounds:              5,000

```

Press Enter to walk through the four phases of one training round...

The learning cycle

The learning cycle repeated in every training round

One round is one complete trip through all four phases. The model repeats this cycle with the same training matrices, but with newly updated weights and biases after each trip.

Phase 1 -> Phase 2 -> Phase 3 -> Phase 4 -> repeat

PHASE 1: FEEDFORWARD PROPAGATION

Question: What prediction does the network produce with its current weights and biases?

Equation path:

$$Z1 = X @ W1 + b1$$

$$A1 = \text{ReLU}(Z1)$$

$$Y_hat = A1 @ W2 + b2$$

Result: The prediction matrix Y_hat .

PHASE 2: LOSS-FUNCTION EVALUATION

Question: How different is the prediction Y_hat from the correct target Y ?

Equation path:

$$error = Y_hat - Y$$

$$J = \text{mean}((Y_hat - Y) ** 2)$$

Result: One scalar loss value J . Smaller is better.

PHASE 3: BACKPROPAGATION

Question: Which weights and biases affected J , and by how much?

Equation path:

$$dJ/dY_hat = (2 / \text{error.size}) * error$$

$$dJ/dW2 = A1.T @ dJ/dY_hat$$

$$dJ/db2 = \text{sum}(dJ/dY_hat)$$

$$dJ/dA1 = dJ/dY_hat @ W2.T$$

$$dJ/dZ1 = dJ/dA1 * \text{ReLU}'(Z1)$$

$$dJ/dW1 = X.T @ dJ/dZ1$$

$$dJ/db1 = \text{sum}(dJ/dZ1)$$

Result: The gradients $dJ/dW1$, $dJ/db1$, $dJ/dW2$, and $dJ/db2$.

PHASE 4: OPTIMIZATION

Question: Given the gradients, how should the weights and biases change?

Equation path:

$$W_new = W_old - eta * dJ/dW$$

```
b_new = b_old - eta * dJ/db
Result: Updated parameters for the next training round.
```

Important distinction: backpropagation calculates gradients; optimization uses those gradients.

Press Enter to begin 5,000 training rounds...

Training the network for 5,000 rounds... complete.

3. REVIEW WHAT CHANGED DURING TRAINING

Loss checkpoints

J is the mean squared error over the standardized training targets. It is one score for the whole training set: a smaller J means the model's predictions are closer to Y, while J = 0 would be a perfect match.

```
J = mean((Y_hat - Y) ** 2)
```

Checkpoint	Loss J	Change from start
Before training	1.1069907030	starting point
After round 1	0.9680029483	12.555458% lower
After round 1,000	0.0001026284	99.990729% lower
After round 2,000	0.0000044341	99.999599% lower
After round 3,000	0.0000004104	99.999963% lower
After round 4,000	0.0000000879	99.999992% lower
After round 5,000	0.0000000361	99.999997% lower

Training summary

```
Initial loss J:          1.1069907030
Final loss J:            0.0000000361
Reduction in J:          99.999997%
Parameter updates completed: 5,000
```

The reduction percentage shows how much the MSE changed over the training process

One visible weight update

```
W1_new = W1_old - eta * dJ/dW1
W1[0, 0] before training: 0.472436
W1[0, 0] after training:  0.483146
```

Press Enter to Next test the model inference using user inputs...

4. TEST THE TRAINED NETWORK

Training is finished. From this point forward, the network performs inference: it uses feedforward propagation to make a prediction, but it does not calculate a training loss, run backpropagation, or update weights.

```
Y_hat = F_theta(X_user_standardized)
```

Build one input row for the trained network

```
X_user = [[x1, x2, x3]]
x1 = highway miles
x2 = local-road miles
x3 = traffic flag: 0 for normal, 1 for congested

Enter x1, highway miles (0 to 30): 24.67
Enter x2, local-road miles (0 to 10): 7.78
Enter x3, traffic condition (0 = normal, 1 = congested): 1
```

5. READ THE ANN PREDICTION

Input supplied to the network

```
X_user = [[24.67, 7.78, 1]]
x1, highway miles:          24.67
x2, local-road miles:       7.78
x3, traffic condition:      1 (congested traffic)
```

```
Y_hat_minutes = Y_hat_standardized * sigma_Y + mu_Y
```

TRAINED ANN PREDICTION

Y_hat ($\hat{y}$) = 92.5487 minutes

Reference check

```
Synthetic reference target Y:      92.5480 minutes
Absolute error |Y_hat - Y|:         0.0007 minutes
The same error in seconds:          0.04 seconds
```

Y_hat is the neural network's answer. The synthetic target Y is shown afterward only so we can check how closely the network learned the fictional classroom rule. Y was not substituted for the prediction.

The trained network produced the highlighted prediction through feedforward propagation using its learned weights and biases.

alex.omegapy@MacStudio CSC510-CTAs %